

# R, RStudio, and the console · vectors · types

Tuesday, May 19, 2026

**i** Today: Tuesday May 19

## Before class

- **First Perusall Assignment:** *Data Computing* §2.1-2.8 ([Click here](#)).

## Plan for today

- Tour of RStudio
- Using R as a calculator
- Data types and vectors
- Creating objects

## Helpful links

- [Syllabus](#) · [AI policy](#) · [Schedule](#) · [Data Computing](#) §2

## What is Computing?

Computing is about turning human ideas into reproducible instructions that a computer can carry out.

In this course, we will learn how to:

- organize data
- automate calculations
- visualize information
- build reproducible analyses
- communicate results

R is the tool we will use to do that.

## Another Introduction to R

- R is a programming language designed for data analysis, statistics, and visualization.
- We use R to tell the computer exactly what computations we want to perform.
- R is widely used in:
  - statistics
  - data science
  - research
  - industry analytics
- R is free, open-source, and highly extensible through thousands of packages that contain additional tools and functions.

## RStudio Desktop (RStudio)

- First version released in 2011.
- *Integrated development environment (IDE)* provides additional tools and window management to R to help coders, statisticians, and data scientists use R more effectively.
- While R is required to run RStudio, you do not need to have RStudio to use R.

## R vs. RStudio

- **R** is the programming language.
- **RStudio** is the environment/interface we use to work with R more comfortably.

# A tour of RStudio

## RStudio's Four Pane Layout

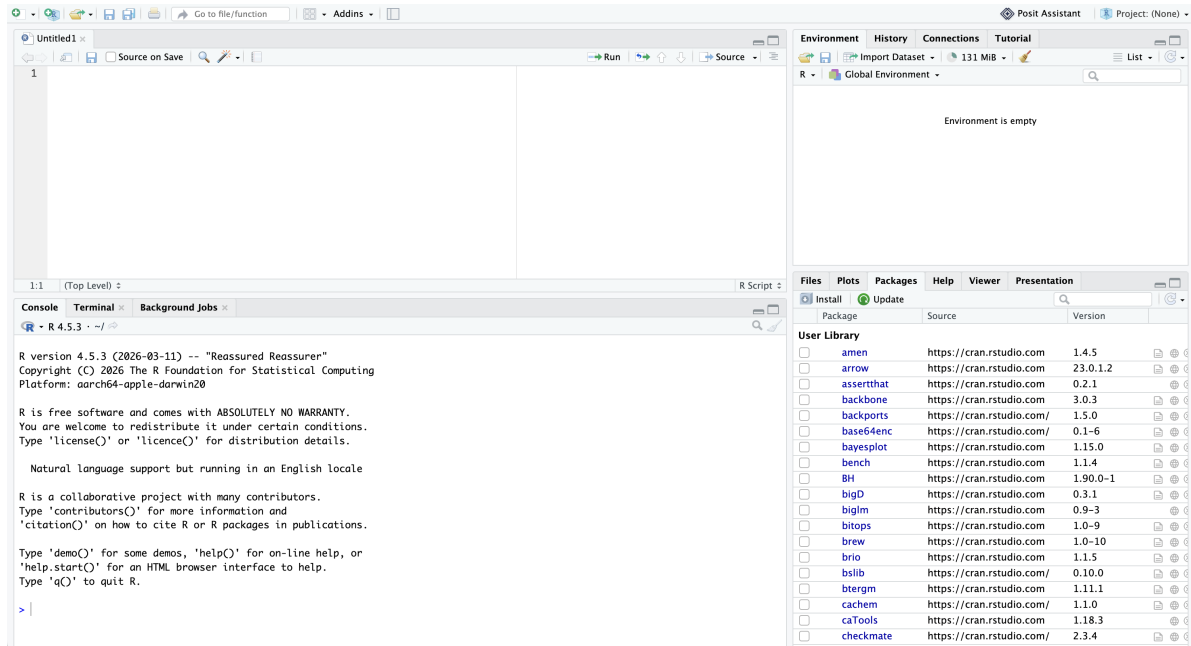


Figure 1: RStudio's four pane layout.

## Source Pane

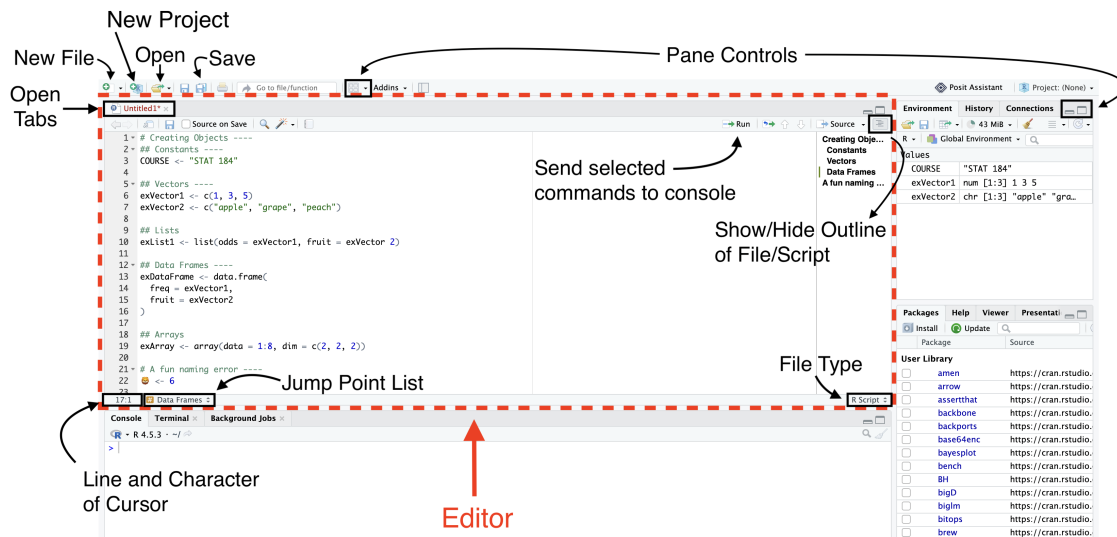


Figure 2: RStudio's source pane, annotated.

## Console Pane

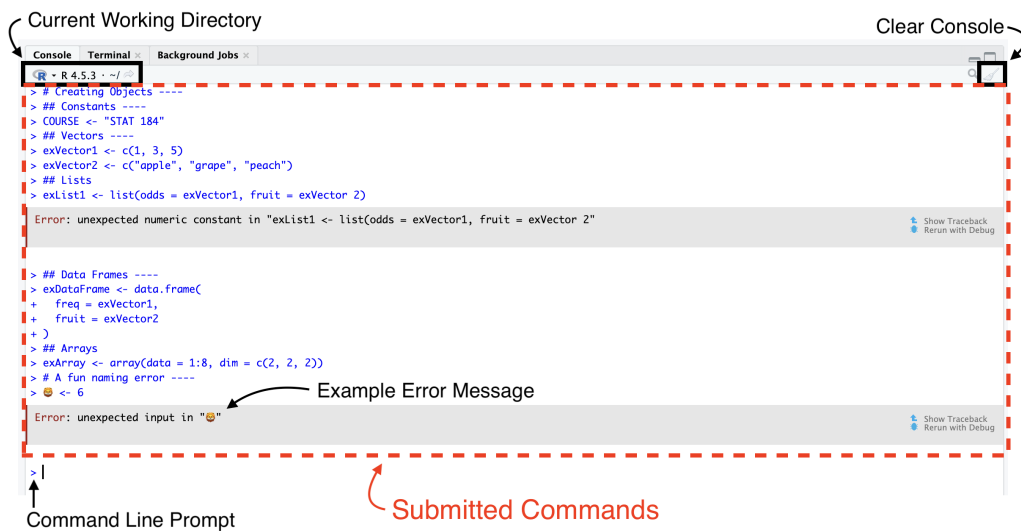


Figure 3: RStudio's console, annotated.

## Environment Pane

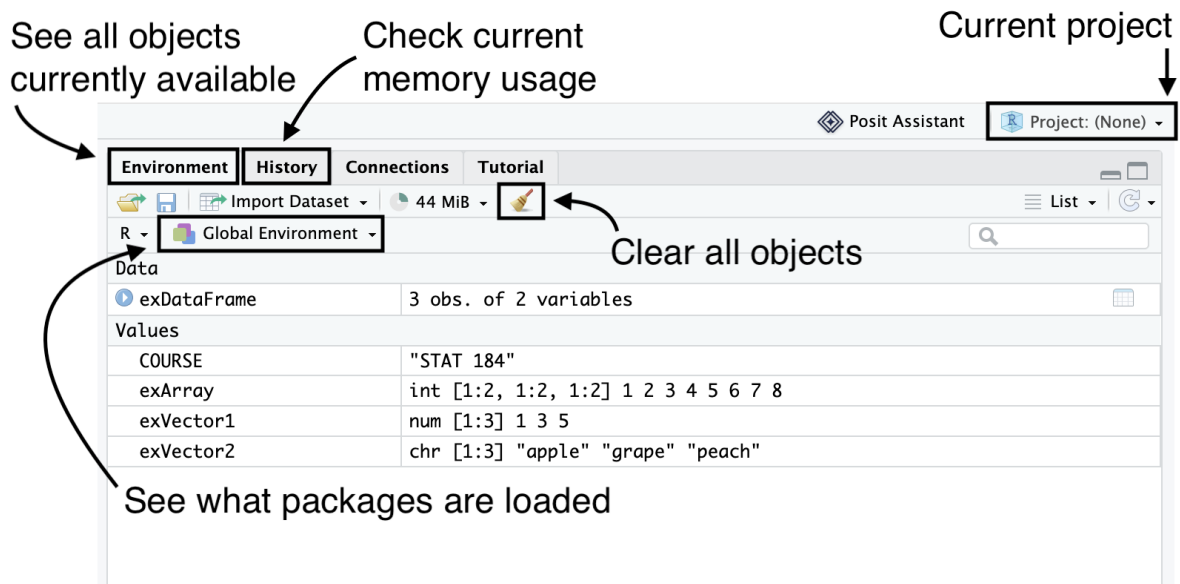


Figure 4: RStudio's environment pane, annotated.

## Output Pane - Files Tab

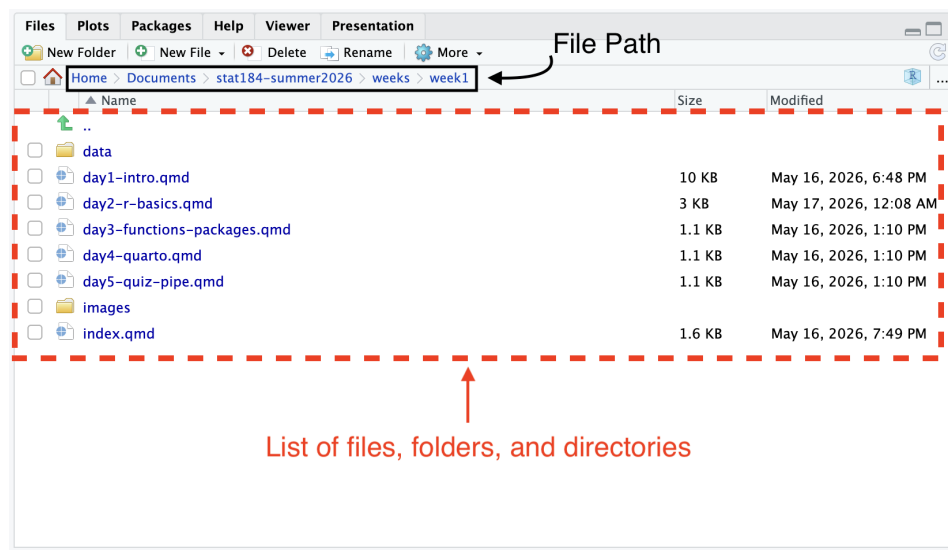


Figure 5: RStudio's output pane (files tab), annotated.

## Output Pane - Plots Tab

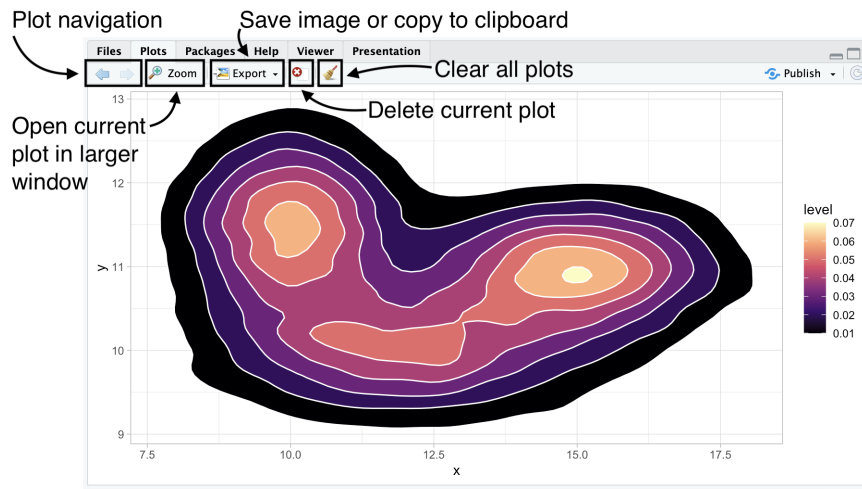


Figure 6: RStudio's output pane (plots tab), annotated, displaying a 2D density contour plot created with ggplot2.

Darker regions indicate higher concentrations of observations; lighter regions represent lower densities. The plot combines filled density polygons with contour boundaries to visualize the distribution of simulated data.

```
# Set Seed
set.seed(184)

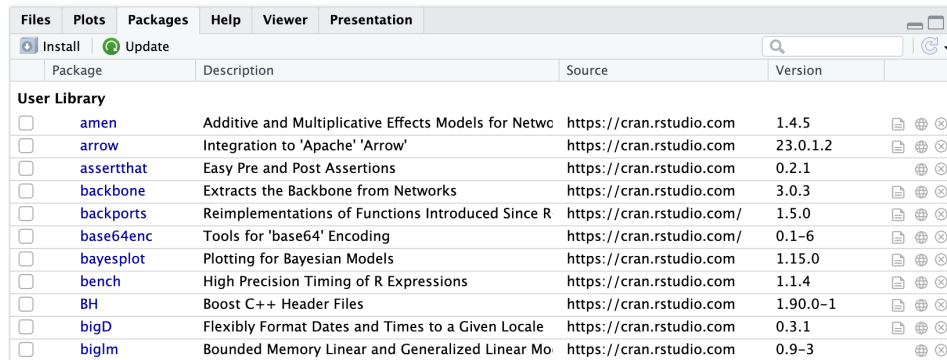
# Library
library(tidyverse)

# Data
a <- data.frame(x = rnorm(20000, 12, 2), y = rnorm(20000, 10, 0.5))
b <- data.frame(x = rnorm(20000, 15, 1.5), y = rnorm(20000, 11, 0.5))
c <- data.frame(x = rnorm(20000, 10, 1.1), y = rnorm(20000, 11.5, 0.7))
data <- rbind(a, b, c)

# Area + contour
ggplot(data, aes(x = x, y = y)) +
  stat_density_2d(aes(fill = after_stat(level)),
    geom = "polygon", colour = "white") +
```

```
theme_light() +
scale_fill_viridis_c(option = "magma")
```

## Output Pane - Packages Tab



| Package                                    | Description                                          | Source                    | Version  |
|--------------------------------------------|------------------------------------------------------|---------------------------|----------|
| <input type="checkbox"/> <b>amen</b>       | Additive and Multiplicative Effects Models for Netwo | https://cran.rstudio.com  | 1.4.5    |
| <input type="checkbox"/> <b>arrow</b>      | Integration to 'Apache' 'Arrow'                      | https://cran.rstudio.com  | 23.0.1.2 |
| <input type="checkbox"/> <b>assertthat</b> | Easy Pre and Post Assertions                         | https://cran.rstudio.com  | 0.2.1    |
| <input type="checkbox"/> <b>backbone</b>   | Extracts the Backbone from Networks                  | https://cran.rstudio.com  | 3.0.3    |
| <input type="checkbox"/> <b>backports</b>  | Reimplementations of Functions Introduced Since R    | https://cran.rstudio.com/ | 1.5.0    |
| <input type="checkbox"/> <b>base64enc</b>  | Tools for 'base64' Encoding                          | https://cran.rstudio.com/ | 0.1-6    |
| <input type="checkbox"/> <b>bayesplot</b>  | Plotting for Bayesian Models                         | https://cran.rstudio.com  | 1.15.0   |
| <input type="checkbox"/> <b>bench</b>      | High Precision Timing of R Expressions               | https://cran.rstudio.com  | 1.1.4    |
| <input type="checkbox"/> <b>BH</b>         | Boost C++ Header Files                               | https://cran.rstudio.com  | 1.90.0-1 |
| <input type="checkbox"/> <b>bigD</b>       | Flexibly Format Dates and Times to a Given Locale    | https://cran.rstudio.com  | 0.3.1    |
| <input type="checkbox"/> <b>biglm</b>      | Bounded Memory Linear and Generalized Linear Mo      | https://cran.rstudio.com  | 0.9-3    |

Figure 7: RStudio's output pane (packages tab).

- The “Packages” tab will list all packages that you have currently installed on your machine. You can load a package by clicking the box to the left of the name.
- You can also use the “Install” and “Update” buttons to help you get new packages and keep them current.

## Output Pane - Help Tab

R: Smoothed density estimates - Find in Topic

geom\_density (ggplot2) R Documentation

### Smoothed density estimates

**Description**

Computes and draws kernel density estimate, which is a smoothed version of the histogram. This is a useful alternative to the histogram for continuous data that comes from an underlying smooth distribution.

**Usage**

```
geom_density(
  mapping = NULL,
```

Figure 8: RStudio's output pane (help tab).

- The “Help” tab displays help documentation that has been included with the packages for functions, data sets, and other objects.

- **Help Shortcuts:** `?barplot`, `help(barplot)`, or `??barplot` (broad sweep). You can also place your cursor in the object name and press `fn + F1` (or just `F1` if your function keys are active).

## Data types and structures

### Data Types

- Data type refers to the nature of how we choose to record some value.
- **Common Data Types in R:**
  - Numeric (Double)
  - Integer
  - Complex
  - Character
  - Logical

### Data Structures

- Data structures are the most important type of base objects in R as they will form the basis for all manipulations we do.
- R's data structures may be organized along two aspects:
  1. How many dimensions we need for values.
  2. If all values have to be the same data type.

Table 1: Summary of R's main base data structures by dimension and type-mixing.

| Structure  | Dimensions   | Mixed types?         |
|------------|--------------|----------------------|
| Vector     | 1            | No                   |
| List       | 1            | Yes                  |
| Matrix     | 2            | No                   |
| Data frame | 2            | Yes (across columns) |
| Array      | $n (\geq 2)$ | No                   |

### Data Structures Examples

Table 2: Examples of each data structure showing its printed form in the R console.

| Dimensions          | All Values Same Data Type                                                                                               | A Mix of Data Types                                      |
|---------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>1 Dimension</b>  | <b>Atomic Vectors</b>                                                                                                   | <b>List</b>                                              |
|                     | [1] 1 3 5                                                                                                               | [[1]]                                                    |
|                     | [1] "apple" "grape"                                                                                                     | [1] 1 3 5                                                |
|                     | "peach"                                                                                                                 | [[2]]                                                    |
|                     |                                                                                                                         | [1] "apple" "grape"                                      |
|                     |                                                                                                                         | "peach"                                                  |
| <b>2 Dimensions</b> | <b>Matrix</b>                                                                                                           | <b>Data Frame</b>                                        |
|                     | <pre>       [,1] [,2] [1,]    1    5 [2,]    3    7 </pre>                                                              | <pre>   id fruit 1  1 apple 2  3 grape 3  5 peach </pre> |
| <b>n Dimensions</b> | <b>Array</b>                                                                                                            | <i>Not applicable</i>                                    |
|                     | <pre> , , 1        [,1] [,2] [1,]    1    5 [2,]    3    7  , , 2        [,1] [,2] [1,]    2    6 [2,]    4    8 </pre> |                                                          |

## Data Structures Visualization

```

# Color palette
bg_color    <- "#FFFFFF"
light_blue  <- "#DDEAF5"
mid_blue    <- "#9BB8DC"
edge_color  <- "#001E44"   # Penn State Navy

# Draw a single unit cube at grid position (gx, gy, gz)
# Uses simple axonometric projection: x' = gx + 0.4*gz, y' = gy + 0.4*gz

```

```

draw_cube <- function(gx, gy, gz, fill = light_blue) {
  s <- 1
  dx <- 0.4 * s
  dy <- 0.4 * s

  px <- gx + dx * gz
  py <- gy + dy * gz

  # Front face
  polygon(c(px, px + s, px + s, px),
          c(py, py, py + s, py + s),
          col = fill, border = edge_color, lwd = 1.2)
  # Top face
  polygon(c(px, px + s, px + s + dx, px + dx),
          c(py + s, py + s, py + s + dy, py + s + dy),
          col = fill, border = edge_color, lwd = 1.2)
  # Right face (slightly darker for shading)
  shade <- adjustcolor(fill, red.f = 0.92, green.f = 0.92, blue.f = 0.95)
  polygon(c(px + s, px + s + dx, px + s + dx, px + s),
          c(py, py + dy, py + s + dy, py + s),
          col = shade, border = edge_color, lwd = 1.2)
}

# Alternating fill in 3D (checkerboard)
checker_fill <- function(i, j, k = 0) {
  if ((i + j + k) %% 2 == 0) light_blue else mid_blue
}

par(mar = c(0, 0, 0, 0), bg = bg_color)
plot.new()
plot.window(xlim = c(0, 18), ylim = c(0, 6), asp = 1)

# 1. Vector: 1 x 3 row
ox <- 1; oy <- 2.2
for (i in 0:2) {
  draw_cube(ox + i, oy, 0, fill = checker_fill(i, 0))
}
text(ox + 1.7, oy - 1, "Vector",
     cex = 1.4, col = edge_color, font = 2)

# 2. Matrix: 3 x 3 grid
ox <- 7; oy <- 1

```

```

for (j in 0:2) {
  for (i in 0:2) {
    draw_cube(ox + i, oy + j, 0, fill = checker_fill(i, j))
  }
}
text(ox + 1.7, oy - 1, "Matrix",
     cex = 1.4, col = edge_color, font = 2)

# 3. Array: 3 x 3 x 3 cube - draw back-to-front, bottom-up, left-right
ox <- 13; oy <- 1
for (k in 2:0) {
  for (j in 0:2) {
    for (i in 0:2) {
      draw_cube(ox + i, oy + j, k, fill = checker_fill(i, j, k))
    }
  }
}
text(ox + 2.3, oy - 1, "Array",
     cex = 1.4, col = edge_color, font = 2)

```

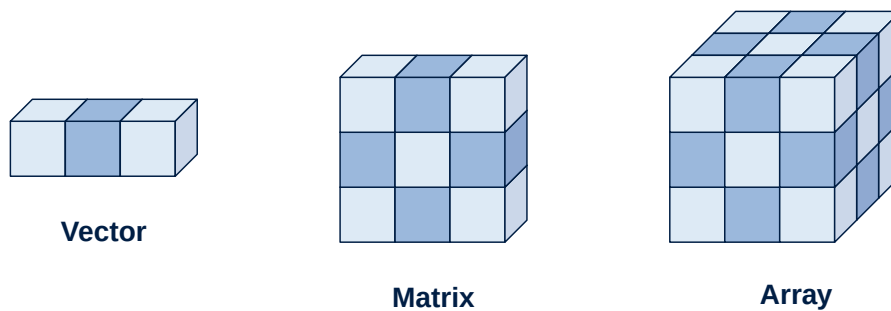


Figure 9: R's data structures as cubes: a vector (1D), matrix (2D), and array (3D).

## R as a calculator · creating objects

### R as a Calculator

You can use R as a calculator:

```
184.101 * 9.35
```

```
[1] 1721.344
```

```
(8 + 4) / 9
```

```
[1] 1.333333
```

```
sqrt(8)
```

```
[1] 2.828427
```

```
cos(2 * pi)
```

```
[1] 1
```

### Your Turn to Use the Calculator

Your turn. Open the console and try each line below; don't just read them, **type them**.

```
2 + 2
```

```
sqrt(16)
```

```
c(1, 2, 3) * 2
```

```
10 %% 3
```

```
x <- log(10^2, base = 10)
```

```
x
```

### Vector Operations

```
c(1, 8, 4) + c(9, 3, 5)
```

```
[1] 10 11 9
```

```
c(1, 8, 4) * c(9, 3, 5)
```

```
[1] 9 24 20
```

- The `c()` function **concatenates** multiple individual values into one single vector.

## Built-in Functions and Constants

```
exp(1)
```

```
[1] 2.718282
```

```
sqrt(pi)
```

```
[1] 1.772454
```

```
dnorm(0)
```

```
[1] 0.3989423
```

- `sqrt` computes the square root.
- `exp` computes the function  $e^x$ .
- `pi` is the constant  $\pi$ .
- `dnorm` computes the standard normal density function.

## Creating objects

You can create new objects with the **assignment operator**, `<-`:

```
x <- 3 * 5
```

The above is an example of an **assignment statement**. In general, assignment statements have the form:

```
object_name <- value
```

In English, this is “object\_name **gets** value.”

- Even though typing <- for the many assignments you will make can be tedious, **don’t use =**; it will work, but will cause confusion later.
- You can use the RStudio keyboard shortcut **Alt + -** (or **Option + -** on a Mac).
- Object names must start with a letter, and can only contain letters, numbers, **\_**, and **.**

## Setup checklist — due by Wednesday

### Install R and RStudio

- R: <https://cloud.r-project.org>
- RStudio: <https://posit.co/download/rstudio-desktop>

 Mac users

Install **XQuartz** too — some packages we’ll use later in the course need it.

### Configure RStudio

- Tools > Global Options... > General
- Restore .RData into workspace at startup: **Unchecked**
- Save workspace to .RData on exit: **Never**

## Wrap-up & looking ahead

**Tomorrow:** Functions, packages, the tidyverse, and objects.

### Upcoming Deadlines

- **Wed 5/20, before class** — [DC §3.1–3.5](#)
- **Wed 5/20, 11:59 p.m.** — [HW 0: Setup](#)
- **Fri 5/22, in class** — Quiz 1
- **Fri 5/22, 11:59 p.m.** — [HW 1: Basic R & Plots](#)

### Reach out

Stuck? Office hours, or email me ([cgc5478@psu.edu](mailto:cgc5478@psu.edu)) or Xinyue ([xpw5228@psu.edu](mailto:xpw5228@psu.edu)).

### Reference

[Syllabus](#) · [AI policy](#) · [Schedule](#) · [Data Computing](#)